NEW COURSE SECTION

DSA + AI

Intelligent Problem Solving in Python

Learn how to use AI as a thinking partner for data structures and algorithms - without giving up the reasoning, correctness checks, or interview skills that matter.

SECTION PROMISE	You will move from problem statement to verified solution using a disciplined loop: understand, plan, implement, test, analyze, and explain.

What you will learn

- Turn a DSA problem into constraints, invariants, examples, and an algorithm plan.
- Use AI for hints, code review, debugging, test generation, and mock interviews.
- Verify AI output with dry runs, unit tests, complexity analysis, and benchmarks.
- Recognize plausible but incorrect code, hidden edge cases, and false complexity claims.

Recommended level	Suggested format
After arrays, strings, hashing, recursion, trees, and graphs.	10 lessons, 4 guided labs, 1 capstone, and 1 practical assessment.

Section at a glance

#	Lesson	Learning outcome	Practice
01	AI's role in DSA	Separate assistance from authorship.	Tool boundaries
02	Problem decomposition	Extract inputs, outputs, constraints, and invariants.	Constraint map
03	Hint-first prompting	Request progressive hints without solution dumping.	Prompt ladder
04	Solution design	Compare brute force and optimized approaches.	Two Sum
05	Debugging	Use traces and counterexamples to find defects.	Binary search
06	Testing	Generate, classify, and verify edge cases.	Unit tests
07	Complexity review	Validate time and space claims.	Growth table
08	Benchmarking	Measure implementations responsibly.	timeit
09	Interview simulator	Explain decisions under follow-up questions.	Mock round
10	Capstone	Produce a verified solution dossier.	Algorithm audit

FOUNDATION

01 The AI-Assisted DSA Workflow

AI is most useful when it strengthens your reasoning loop. It should not replace the loop.

1. Understand

Restate the problem. Identify input, output, constraints, examples, and ambiguous wording.

2. Plan

Write a brute-force idea, then identify the bottleneck and a suitable data structure.

3. Ask

Request a hint, critique, counterexample, or comparison - not an unexplained final answer.

4. Implement

Write the solution yourself. Keep names, invariants, and boundary conditions explicit.

5. Verify

Dry-run examples. Test empty, minimal, duplicate, sorted, invalid, and large inputs.

6. Analyze

Derive time and space complexity from operations and data-structure behavior.

7. Explain

State the approach, proof idea, complexity, tradeoffs, and alternatives in your own words.

RULE OF OWNERSHIP If you cannot explain why each important line exists, the solution is not yet yours. Ask AI to question you until you can defend the algorithm.

Choose the right kind of AI help

Need	Ask AI for	You still verify
Stuck at the start	One conceptual hint	Whether the hint fits constraints
Two approaches	Tradeoff comparison	Actual operation counts and memory
Wrong output	Small counterexample and trace	First point where state diverges
Solution seems slow	Bottleneck review	Complexity derivation and benchmark
Interview practice	Follow-up questions	Your explanation without AI text

CORE SKILL

02 Prompting for Reasoning, Not Shortcuts

A strong DSA prompt includes the problem, constraints, your current thinking, and the exact kind of help you want. This makes the response easier to inspect and less likely to skip important assumptions.

The HINT framework

H - Hand over context

Provide the full problem, input format, output format, and constraints.

I - Include your attempt

Share pseudocode, code, dry run, or the exact point where you are stuck.

N - Name the help

Ask for a hint, critique, trace, counterexample, proof, test cases, or complexity review.

T - Test the response

Challenge assumptions and independently verify every important claim.

PROMPT PATTERN

```
Problem: [paste the problem]
Constraints: [n range, value range, memory or ordering limits]
My approach: [your current idea or code]
Help needed: Give me one hint only. Do not provide complete code.
Then ask me a question that checks whether I understood the hint.
```

PROMPT PATTERN

```
Review this algorithm as a strict code reviewer.
1. State the invariant it appears to rely on.
2. Find the smallest failing input, if one exists.
3. Trace only that input in a compact table.
4. Do not rewrite the code until I explain the bug.
```

PROMPT PATTERN

```
Compare Approach A and Approach B for the stated constraints.
Derive time and auxiliary space complexity step by step.
Call out assumptions about Python list, dict, set, and sorting operations.
Finish with one case where the slower-looking approach may still be preferable.
```

AVOID VAGUE PROMPTS Instead of 'solve this,' request a specific cognitive action: clarify, hint, challenge, trace, test, analyze, compare, or interview.

Progressive hint ladder

- Level 1: Ask what pattern or data structure may apply.
- Level 2: Ask for the key invariant or state to maintain.
- Level 3: Ask for pseudocode with no language-specific syntax.
- Level 4: Share your code and ask for the smallest counterexample.
- Level 5: Request a reference solution only after completing your own attempt.

FROM BRUTE FORCE TO HASHING

03 Worked Example: Two Sum

Problem: Given a list of integers and a target, return the indices of two different elements whose values add to the target. Assume exactly one valid pair exists.

THINK FIRST What must be remembered while scanning the list? For each value x, the required partner is target - x.

Approach A - brute force

```
def two_sum_brute(nums, target):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return [i, j]
    return []
```

The nested search checks pairs directly. In the worst case it performs a quadratic number of comparisons: O(n^2) time and O(1) auxiliary space.

Approach B - one-pass hash map

```
def two_sum(nums, target):
    seen = {}
    for index, value in enumerate(nums):
        needed = target - value
        if needed in seen:
            return [seen[needed], index]
        seen[value] = index
    return []
```

With average-case constant-time dictionary lookup and insertion, the scan is O(n) time and uses O(n) auxiliary space.

Invariant

Before processing index i, seen stores values from indices smaller than i. Therefore, a match found in seen always uses two different elements.

i	value	needed	seen before	action
0	2	7	{}	store 2:0
1	7	2	{2: 0}	return [0, 1]

Ask AI to challenge the solution

PROMPT PATTERN

```
For this Two Sum implementation, do not rewrite the code.
Generate six test cases covering duplicates, negative values, zero, a pair near the end, and minimal input.
For each case, state which behavior it verifies.
```

VERIFICATION

04 Debugging and Edge-Case Testing

AI can accelerate debugging when you ask for evidence: a small failing input, a state trace, and the first violated invariant.

Bug hunt - binary search

```
def binary_search(nums, target):
    left, right = 0, len(nums) - 1
    while left < right:             # Bug: stops before checking one candidate
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        if nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

SMALLEST COUNTEREXAMPLE nums = [5], target = 5. The loop never runs because left == right, so the function incorrectly returns -1.

Correct boundary condition

```
def binary_search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        if nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Build a test matrix before generating test code

Category	Example	Why it matters
Smallest valid	[5], target 5	Catches skipped final candidate
Absent target	[1, 3, 7], target 4	Checks clean termination
First position	[2, 4, 6], target 2	Exercises left boundary
Last position	[2, 4, 6], target 6	Exercises right boundary
Duplicates	[2, 2, 2], target 2	Checks equality handling

Convert the matrix into repeatable tests

```
import unittest

class TestBinarySearch(unittest.TestCase):
    def test_single_item(self):
        self.assertEqual(binary_search([5], 5), 0)

    def test_boundaries(self):
        self.assertEqual(binary_search([2, 4, 6], 2), 0)
        self.assertEqual(binary_search([2, 4, 6], 6), 2)

    def test_absent(self):
        self.assertEqual(binary_search([1, 3, 7], 4), -1)
```

ANALYSIS

05 Complexity Claims Need Proof

AI may state a complexity confidently without accounting for nested work, copying, sorting, recursion depth, or Python container behavior. Derive the claim before accepting it.

A five-question complexity audit

- How many times does each loop or recursive call execute?
- What is the cost of the operation inside the loop?
- Does any slicing, copying, sorting, or string concatenation happen?
- What data grows with input size, including call stack and temporary structures?
- Is the claim worst-case, average-case, amortized, or expected?

Pattern	Typical time	Typical extra space	Verification cue
Single scan	O(n)	O(1) or O(n)	Inspect stored state
Nested full scans	O(n^2)	Often O(1)	Multiply loop counts
Divide search range	O(log n)	O(1) iterative	Range halves each step
Sort then scan	O(n log n)	Implementation-dependent	Include sort cost
Graph traversal	O(V + E)	O(V)	Visit each vertex and edge

Benchmark after analysis

Benchmarking supports an analysis; it does not replace asymptotic reasoning. Use the same inputs, run several repetitions, and avoid timing setup work when possible.

```
from timeit import repeat

nums = list(range(2_000))
target = 3_997

brute_times = repeat(
    lambda: two_sum_brute(nums, target), repeat=5, number=10
)
hash_times = repeat(
    lambda: two_sum(nums, target), repeat=5, number=10
)

print("best brute:", min(brute_times))
print("best hash: ", min(hash_times))
```

INTERPRET CAREFULLY A faster result on one input size does not prove a better Big-O class. Constant factors, data distribution, interpreter behavior, and hardware affect measured time.

PROMPT PATTERN

```
Audit my complexity analysis. Identify every operation whose cost I may have hidden.
Separate worst-case time, expected time, auxiliary space, and output space.
Do not provide a corrected answer until you show the disputed step.
```

COMMUNICATION

06 Turn AI into a DSA Interviewer

Interview performance depends on how clearly you explore the problem, justify tradeoffs, and respond to changing constraints. Use AI to create pressure and follow-up questions, not to recite an answer.

Interview simulation prompt

PROMPT PATTERN

```
Act as a technical interviewer for a Python DSA round.
Give me one medium-level problem involving [topic].
Reveal no solution. Ask one question at a time.
Evaluate my clarification, brute-force idea, optimization, code, tests,
and complexity. If I am stuck, offer only one small hint.
At the end, score me using the rubric below.
```

Recommended speaking sequence

1. Clarify	2. Example
Confirm inputs, outputs, duplicates, ordering, invalid cases, and constraints.	Create a small example and state the expected result.
3. Baseline	**4. Optimize**
Describe a correct brute-force approach before optimizing.	Name the bottleneck and the data structure or pattern that removes it.
5. Code	**6. Validate**
Explain invariants and boundary choices while implementing.	Dry-run normal and edge cases; state time and space complexity.

Self-evaluation rubric

Skill	1 - Developing	2 - Competent	3 - Strong
Problem framing	Misses key constraints	Clarifies main cases	Surfaces assumptions and risks
Algorithm	Needs solution rescue	Finds correct approach	Compares alternatives clearly
Correctness	Few tests	Tests normal and edge cases	Uses invariants and counterexamples
Complexity	States unsupported claim	Derives main costs	Qualifies cases and tradeoffs
Communication	Hard to follow	Explains key decisions	Clear, concise, and adaptive

PRACTICE WITHOUT DEPENDENCE	Complete at least one uninterrupted solve each week with AI closed. Then compare your reasoning with an AI-assisted review.

PRACTICE

07 Hands-On Labs and Capstone

Each lab produces evidence of reasoning, not only a final code file.

Lab 1	Prompt Ladder Choose an array or string problem. Solve it using progressively stronger hints. Record which hint unlocked the solution and why.

Lab 2	Counterexample Hunt Give AI an intentionally flawed sliding-window or binary-search solution. Validate its counterexample and explain the violated invariant.
Lab 3	Test Designer Create a test matrix for a stack, queue, tree, or graph problem. Convert the matrix to unittest cases.
Lab 4	Performance Review Compare two correct approaches. Derive complexity, benchmark several input sizes, and explain any mismatch.

Capstone - Verified Algorithm Dossier

Select one medium-level problem from arrays, linked lists, trees, graphs, dynamic programming, or greedy algorithms. Submit one concise dossier containing all items below.

Deliverable	What it must show
1. Problem map	Inputs, outputs, constraints, assumptions, and examples
2. Baseline solution	Correct brute-force idea and its bottleneck
3. Optimized solution	Python code with a named invariant or proof idea
4. AI interaction log	Three useful prompts and what you accepted, rejected, or corrected
5. Verification pack	Dry run, edge-case matrix, and automated tests
6. Analysis	Time, auxiliary space, tradeoffs, and benchmark notes
7. Reflection	What AI improved and what still required human judgment

Assessment weighting

30%	25%	25%	20%
Correctness and edge-case coverage	Algorithm reasoning and complexity	Code quality and verification	AI judgment and reflection

FINISH STRONG

08 Responsible Use and Reference Sheet

AI-generated code is a draft until it has been understood, tested, and reviewed. Protect private data, respect assessment rules, and disclose AI assistance when required.

Green, amber, red

GREEN - Good use	AMBER - Use carefully	RED - Do not outsource
Hints, Socratic questions, test ideas, traces, alternative approaches, complexity critique, interview practice.	Generated code, refactoring, benchmark interpretation, library choices, and confident complexity claims.	Submitting unverified code, copying hidden assessments, exposing private data, or claiming work you cannot explain.

Final verification checklist

[ ] I can restate the problem and constraints without AI.

[ ] I can explain the chosen data structure and invariant.

[ ] I manually traced at least one normal and one edge case.

[ ] I tested minimal, boundary, duplicate, absent, and large-input behavior where relevant.

[ ] I derived time and auxiliary space complexity from operations.

[ ] I checked any AI claim that depends on Python behavior.

[ ] I can modify the solution when the interviewer changes a constraint.

[ ] I followed course rules for disclosure and academic integrity.

Official resources

Resource	Use it for
Python tutorial - Data Structures	Lists, stacks, queues, comprehensions, tuples, sets, dictionaries, and looping techniques.
Python unittest documentation	Test cases, assertions, fixtures, test suites, and command-line discovery.
Python timeit documentation	Reliable timing of small Python code snippets with callable and command-line interfaces.

TAKEAWAY Use AI to increase the quality of your questions, evidence, and explanations. The goal is not faster copying; it is stronger algorithmic judgment.